Cloudflare Tunnel — Complete Setup Documentation

Secure Public Exposure of a Self-Hosted Homelab Application

System: Ubuntu Server 22.04 LTS
Environment: Proxmox VE 8.x — VM3 (lb-vm,)
Domain: (registered at)
Tunnel Name:
Role: Expose internal NGINX load balancer to the internet securely

---

Variables — Adapt Before Use

```
YOUR_DOMAIN=<your-domain>              # e.g. example.com
TUNNEL_NAME=<your-tunnel-name>         # e.g. homelab
LB_IP=<your-lb-vm-ip>                  # e.g. 192.168.x.x
APP_SERVER_IP=<your-app-vm-ip>         # e.g. 192.168.x.x
MONITORING_IP=<your-monitoring-vm-ip>  # e.g. 192.168.x.x
USER=<your-username>
TUNNEL_ID=<generated-during-setup>
CF_NAMESERVER_1=<assigned-by-cloudflare>
CF_NAMESERVER_2=<assigned-by-cloudflare>
YOUR_REGISTRAR=<your-domain-registrar>
```

Table of Contents

1. Architecture Overview
2. Domain Registration & Setup

3. Cloudflare Account & Domain Configuration
4. DNS Nameserver Migration
5. cloudflared Installation
6. Tunnel Authentication
7. Tunnel Creation
8. Tunnel Configuration
9. DNS Routing
10. System Service Setup
11. NGINX Security Headers
12. Verification & Results
13. Troubleshooting Log

---

1. Architecture Overview

Traffic Flow

```
Internet User
      ↓
https://<YOUR_DOMAIN> (HTTPS, port 443)
      ↓
Cloudflare Edge Network
  - DDoS protection
  - SSL/TLS termination
  - IP masking
  - Caching
      ↓
Cloudflare Tunnel (outbound-only, no open ports)
      ↓
cloudflared daemon (VM3 — lb-vm, <LB_IP>)
      ↓
NGINX Load Balancer (localhost:80)
      ↓
Flask Application (VM1 — app-vm, <APP_SERVER_IP>:80)
      ↓
PostgreSQL + Redis (Docker internal network)
```

Why Cloudflare Tunnel vs Traditional Reverse Proxy

Aspect	Traditional (port forwarding)	Cloudflare Tunnel
Open ports on router	Required (80, 443)	None
Public IP exposure	Yes	No
SSL certificate management	Manual (Certbot + cron)	Automatic
DDoS protection	None	Built-in
IP masking	No	Yes
Cost	Free (Certbot)	Free
Setup complexity	High	Low
Works behind double NAT	No	Yes

> *This homelab runs behind a **double NAT** setup (Altice Labs ISP router → Huawei lab gateway), making traditional port forwarding impossible without ISP cooperation. Cloudflare Tunnel bypasses this completely via outbound-only connections.*

---

2. Domain Registration & Setup

Registrar:

Domain purchased: `<YOUR_DOMAIN>`
TLD: `.fr`

Registration cost: first year
Renewal cost: /year

Post-Purchase Cleanup

bundles a **Zimbra Email Account** (/month) with domain registration. This was cancelled immediately after purchase:

1. dashboard → **My offers and services**
2. Found: `Zimbra Email Account` — Automatic renewal, Every month
3. Clicked ··· → **Cancel service**
4. Selected reason: **Order errors/double orders**
5. Selected: **I do not plan to replace this service**
6. Confirmed cancellation → Status changed to **Termination requested**

> *WARNING: **Always check for bundled services after domain registration.** Registrars commonly add email accounts, SSL certificates, or hosting plans to orders without clear disclosure. Cancel anything you don't need immediately to avoid charges.*

Services Retained

Service	Type	Renewal	Cost
	Domain	Yearly	/year
	DNS zones	Yearly	€7-1000
MX Plan	Email redirect	None	Free
~~Zimbra Email Account~~	~~Email~~	~~Monthly~~	~~Cancelled~~

---

3. Cloudflare Account & Domain Configuration

Account Setup

1. Created free account at `cloudflare.com`
2. Clicked **Add a site**
3. Entered `<YOUR_DOMAIN>`
4. Selected **Free plan**
5. Selected **Import DNS records automatically**

DNS Records Cleanup

Cloudflare scanned and imported DNS records. All -specific records were deleted since they are no longer needed:

Record	Type	Content	Action
	A	213.186.33.5 ()	Deleted
www	A	213.186.33.5 ()	Deleted
ftp	CNAME		Deleted
	MX	mx1.mail.ovh.net	Deleted
	MX	mx2.mail.ovh.net	Deleted
	MX	mx3.mail.ovh.net	Deleted
	TXT	SPF record	Deleted
	TXT	verification	Deleted
www	TXT	welcome	Deleted

> *NOTE: DNS records are left empty at this stage. Cloudflare Tunnel creates its own CNAME records automatically in the next steps.*

Assigned Cloudflare Nameservers

```
<CF_NAMESERVER_1>
<CF_NAMESERVER_2>
```

---

4. DNS Nameserver Migration

Update Nameservers at

1. dashboard → `<YOUR_DOMAIN>` → **DNS servers**
2. Clicked **Edit DNS** → Selected **Use my own DNS**
3. Added nameservers:
   - `<CF_NAMESERVER_1>` (no associated IP needed)
   - `<CF_NAMESERVER_2>` (no associated IP needed)
4. Clicked **Apply configuration**

Verification

confirmed both nameservers as **Active**:

```
DNS servers          Status    Type
<CF_NAMESERVER_1>    Active    Custom
<CF_NAMESERVER_2>     Active    Custom
```

Propagation

- Nameserver changes propagated within **~5 minutes**

- Cloudflare sent email confirmation: **" is now active on Cloudflare"**

> *NOTE: DNS propagation typically takes 5 minutes to 24 hours depending on TTL values. 's default TTL is low enough that changes propagate quickly.*

---

5. cloudflared Installation

Add Cloudflare APT Repository

```
curl -L https://pkg.cloudflare.com/cloudflare-main.gpg | sudo tee
/usr/share/keyrings/cloudflare-main.gpg > /dev/null

echo "deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg]
https://pkg.cloudflare.com/cloudflared jammy main" \
  | sudo tee /etc/apt/sources.list.d/cloudflared.list
```

Install

```
sudo apt update && sudo apt install cloudflared -y
```

Verify

```
cloudflared --version
```

---

6. Tunnel Authentication

```
cloudflared tunnel login
```

What this does:

1. Outputs a Cloudflare authentication URL
2. Opening the URL in browser shows the **Authorize Cloudflare Tunnel** page
3. Selected `<YOUR_DOMAIN>` from the domain list
4. Authorized the connection

Result:

```
You have successfully logged in.
Credentials saved to: /home/<USER>/.cloudflared/cert.pem
```

The `cert.pem` file authorizes `cloudflared` to create and manage tunnels on the `<YOUR_DOMAIN>` zone.

---

7. Tunnel Creation

```
cloudflared tunnel create <TUNNEL_NAME>
```

Result:

```
Created tunnel homelab with id <TUNNEL_ID>
Tunnel credentials written to /home/<USER>/.cloudflared/<TUNNEL_ID>.json
```

> *WARNING: The `<TUNNEL_ID>.json` file contains the tunnel credentials. It is kept in `/etc/cloudflared/` and never committed to version control.*

List Tunnels

```
cloudflared tunnel list
```

8. Tunnel Configuration

Create Configuration File

```
nano ~/.cloudflared/config.yml
```

Configuration

```
tunnel: <TUNNEL_ID>
credentials-file: /home/<USER>/.cloudflared/<TUNNEL_ID>.json

ingress:
  - hostname: <YOUR_DOMAIN>
    service: http://localhost:80
  - hostname: www.<YOUR_DOMAIN>
    service: http://localhost:80
  - service: http_status:404
```

Ingress rules explained:

- Rule 1: Route `<YOUR_DOMAIN>` traffic to NGINX on `localhost:80`
- Rule 2: Route `www.<YOUR_DOMAIN>` traffic to NGINX on `localhost:80`
- Rule 3: Catch-all rule — required by cloudflared, returns 404 for unmatched hostnames

Copy Files for System Service

```
sudo mkdir -p /etc/cloudflared
sudo cp ~/.cloudflared/config.yml /etc/cloudflared/config.yml
sudo cp ~/.cloudflared/<TUNNEL_ID>.json /etc/cloudflared/
sudo cp ~/.cloudflared/cert.pem /etc/cloudflared/
```

Update credentials-file Path

```
sudo nano /etc/cloudflared/config.yml
```

Updated `credentials-file` to use absolute system path:

```
credentials-file: /etc/cloudflared/<TUNNEL_ID>.json
```

---

9. DNS Routing

Create CNAME records pointing the domain to the tunnel:

```
cloudflared tunnel route dns <TUNNEL_NAME> <YOUR_DOMAIN>
cloudflared tunnel route dns <TUNNEL_NAME> www.<YOUR_DOMAIN>
```

What this creates in Cloudflare DNS:

Name	Type	Content	Proxy
	CNAME	`<TUNNEL_ID>.cfargotunnel.com`	Proxied
www	CNAME	`<TUNNEL_ID>.cfargotunnel.com`	Proxied

> *NOTE: The **Proxied** (orange cloud) status is critical — it means traffic goes through Cloudflare's network, enabling DDoS protection, SSL termination, and IP masking.*

---

10. System Service Setup

Install as systemd Service

```
sudo cloudflared service install
sudo systemctl enable cloudflared
sudo systemctl start cloudflared
```

Verify

```
sudo systemctl status cloudflared
```

Expected output:

```
● cloudflared.service
   Active: active (running)
```

Service behavior:

- ○ Starts automatically on system boot
- ○ Restarts automatically if it crashes
- ○ Runs in background permanently
- ○ Managed by systemd

---

11. NGINX Security Headers

Configuration

Added security headers to `/etc/nginx/sites-available/loadbalancer.conf`:

```
upstream app_servers {
    server <APP_SERVER_IP>:80;
}

server {
    listen 80;
    server_name _;
```

```
    # Security Headers
    add_header X-Frame-Options "DENY";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";
    add_header Content-Security-Policy "default-src 'self'; style-src
'self';";
    add_header Referrer-Policy "strict-origin-when-cross-origin";
    add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
    add_header Permissions-Policy "geolocation=(), microphone=(), camera=
()";

    location / {
        proxy_pass http://app_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Header Explanations

Header	Value	Purpose
`X-Frame-Options`	`DENY`	Prevents clickjacking — page cannot be embedded in iframes
`X-Content-Type-Options`	`nosniff`	Prevents MIME type sniffing attacks
`X-XSS-Protection`	`1; mode=block`	Enables browser XSS filter
`Content-Security-Policy`	`default-src 'self'`	Restricts resource loading to same origin only

Header	Value	Purpose
`Referrer-Policy`	`strict-origin-when-cross-origin`	Controls referrer information sent with requests
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	Forces HTTPS for 1 year (31,536,000 seconds = 60×60×24×365)
`Permissions-Policy`	`geolocation=(), microphone=(), camera=()`	Disables browser feature access

> *NOTE: **HSTS deep dive:** `max-age=31536000` tells browsers to automatically use HTTPS for all requests to this domain for exactly 1 year — without ever attempting HTTP first. This eliminates the risk of SSL stripping attacks on subsequent visits. `includeSubDomains` extends this protection to all subdomains.*

Apply

```
sudo nginx -t && sudo systemctl reload nginx
```

Result

Scanned with `securityheaders.com`:

```
Grade: A+
Headers present:
  [+] Content-Security-Policy
  [+] Referrer-Policy
  [+] X-Content-Type-Options
  [+] X-Frame-Options
```

```
  [+] Strict-Transport-Security
  [+] Permissions-Policy
```

---

12. Verification & Results

Public Access

URL	Result
`https://<YOUR_DOMAIN>`	App loads with valid SSL
`https://www.<YOUR_DOMAIN>`	App loads with valid SSL
`http://<YOUR_DOMAIN>`	Auto-redirected to HTTPS

SSL Certificate

- **Issuer:** Cloudflare
- **Type:** Automatic (no manual renewal needed)
- **Protocol:** TLS 1.3

Security Score

- **securityheaders.com:** A+
- **IP exposed:** No (Cloudflare IP shown, not homelab IP)
- **Open ports on router:** None

Tunnel Status

```
cloudflared tunnel info <TUNNEL_NAME>
```

```
sudo systemctl status cloudflared
# Active: active (running)
```

13. Troubleshooting Log

Issue: CSS returning 404 after CSP headers added

Root cause: Cloudflare was caching the 404 response for `/static/style.css` from before the static files were properly configured.

Resolution:

1. Cloudflare dashboard → **Caching** → **Configuration**
2. **Purge Cache** → **Purge Everything**
3. Hard refresh browser (`Ctrl + Shift + R`) or test in incognito

Lesson: Always purge Cloudflare cache after making changes to static assets or server configuration.

Issue: Inline styles blocked by CSP

Root cause: `Content-Security-Policy: default-src 'self'` blocks all inline `style=""` attributes and `<style>` tags.

Resolution:

1. Moved all CSS from `<style>` tags to external `.css` files (`static/style.css` , `static/auth.css`)
2. Replaced all inline `style=""` attributes with CSS classes
3. Updated CSP to `default-src 'self'; style-src 'self';`

Lesson: Never use inline styles when CSP is enforced. Always use external stylesheets referenced via `url_for('static', filename='...')` in Flask.

Issue: Tunnel service fails to install — config not found

Root cause: `cloudflared service install` looks for config in `/etc/cloudflared/` by default, but the config was only in `~/.cloudflared/`.

Resolution:

```
sudo mkdir -p /etc/cloudflared
sudo cp ~/.cloudflared/config.yml /etc/cloudflared/
sudo cp ~/.cloudflared/<TUNNEL_ID>.json /etc/cloudflared/
sudo cp ~/.cloudflared/cert.pem /etc/cloudflared/
```

Then update `credentials-file` in `/etc/cloudflared/config.yml` to use the `/etc/cloudflared/` path.